CS-4032: Web Programming

Assignment #02

2D Cricket Web Application

Name	Fatima Farrukh Rana
Roll Number	23i-0773
Section	E
Course	CS-4032: Web Programming
Assignment	#02 — 2D Cricket Web Application
Date	26 March 2026

Technologies Used: React 18 | HTML5 Canvas | CSS3 | JavaScript

1. Game Logic and Working Mechanism

Cricket Blitz is a single-player 2D cricket batting game built with React 18, HTML5 Canvas, CSS3, and JavaScript. The player acts as a batsman and faces 2 overs (12 balls total) with a maximum of 2 wickets. On each ball the player selects a batting style and interacts with a probability-based power bar to determine the shot outcome. No random number generation is used — the result depends entirely on where the slider is when the player clicks.

1.1 Game Flow

Step 1 — Select Style: Before each ball, the player chooses Aggressive (high risk/reward) or Defensive (low risk/reward). The style can only be changed between balls.

Step 2 — Click BOWL!: The bowling animation starts. The ball travels from the bowler's end (right side) toward the batsman (left side) over 900ms using requestAnimationFrame on an HTML5 Canvas.

Step 3 — Power Bar Activates: Once the ball arrives, the power bar slider begins moving continuously back and forth across the coloured probability segments. Speed varies slightly per ball.

Step 4 — Click to Play Shot: The player clicks the power bar canvas. The slider position at that exact moment determines the outcome — it maps directly to whichever probability segment the slider is inside.

Step 5 — Result Displayed: The batting animation triggers (bat swings, ball flies), commentary is shown, and the scoreboard updates immediately.

Step 6 — Repeat / Game Over: The game ends when 12 balls are bowled or 2 wickets are lost, showing a full innings summary with rating.

1.2 Game Rules

- **Total overs: 2 (12 balls)**
- Total wickets: 2
- Each ball = one batting attempt
- Game ends when all overs are complete OR all wickets are lost
- Restart option resets: Runs, Wickets, Overs, Balls, and Batting Style

1.3 React State Management

All game state is managed in a custom React hook (useGameState.js) using useState and useCallback. The game phases are: idle → bowling → batting → result → gameover. Each transition is controlled by the hook's action functions (startBowling, startBatting, playShot, nextBall, restartGame). No state mutation happens outside the hook, ensuring a clean separation of concerns.

2. Mapping of Probabilities to the Power Bar

Each batting style defines a discrete probability distribution over 7 outcomes. All probabilities in each style sum exactly to 1.00. The power bar is divided into proportional colour-coded segments matching these probabilities. The outcome is determined by which segment the slider occupies at click time — no random number generation is involved.

2.1 Probability Distributions

Outcome	Aggressive	Bar Segment	Defensive	Bar Segment
Wicket	30%	0.00 – 0.30	12%	0.00 – 0.12
0 Runs	8%	0.30 – 0.38	25%	0.12 – 0.37
1 Run	10%	0.38 – 0.48	30%	0.37 – 0.67
2 Runs	12%	0.48 – 0.60	18%	0.67 – 0.85
3 Runs	5%	0.60 – 0.65	8%	0.85 – 0.93
4 Runs	15%	0.65 – 0.80	5%	0.93 – 0.98
6 Runs	20%	0.80 – 1.00	2%	0.98 – 1.00
TOTAL	**100%**		**100%**	

2.2 How Segments Map to the Bar

The buildSegments() function in gameConfig.js iterates through the probability array for the selected style and accumulates a running cumulative total. Each segment is assigned a start and end value. These values directly control the pixel width of each colour region on the Canvas-rendered power bar.

Example (Aggressive style): If the slider stops at position 0.72, it falls inside the 4 Runs segment (0.65–0.80), so the result is FOUR. If it stops at 0.15, it is in the Wicket region (0.00–0.30), so the result is OUT.

2.3 Style Comparison

- Aggressive — Wicket risk: 30% vs Defensive: 12%. Boundary (4+6) chance: 35% vs 7%.
- Defensive — Lower wicket risk, higher singles/dots. Suited to preserving wickets.
- Both styles sum exactly to 1.00 — verified in gameConfig.js with cumulative accumulation.

3. Implementation of Animations

3.1 Bowling Animation (CricketField.jsx)

When gamePhase transitions to "bowling", a requestAnimationFrame loop starts inside a useEffect. The ball's (x, y) coordinates are linearly interpolated from the bowler's stump position (450, 90) to the batsman position (240, 180) over 900ms. Progress is calculated as elapsed / 900, clamped to 1. A bowlingDoneFired boolean ref ensures the onBowlingDone callback fires exactly once when progress reaches 1.0, preventing double-triggers and phase bugs.

3.2 Batting Animation (CricketField.jsx)

When gamePhase becomes "result", the bat rotation angle is animated from -0.3 radians to +1.7 radians over 300ms using ctx.save() / ctx.rotate() / ctx.restore() on the Canvas 2D context. Simultaneously the ball's position is offset in the +X and -Y direction scaled by elapsed time, simulating it flying toward the boundary. The animation continues running until the next phase begins.

3.3 Power Bar Slider (PowerBar.jsx)

PowerBar.jsx runs two nested requestAnimationFrame loops. The outer loop updates sliderRef.current at a speed of 0.007–0.013 per frame, bouncing the value between 0 and 1 by flipping a direction ref. Speed is randomised per ball for progressive difficulty. The inner loop (in PowerBarVisual) re-renders the Canvas every frame, reading sliderRef.current directly via a ref (not state) to achieve zero-lag rendering without triggering React re-renders.

3.4 Result Badge & Commentary Animations (CSS)

The result badge uses a CSS @keyframes animation (badge-pop) with a cubic-bezier spring curve for a satisfying pop effect. Wicket outcomes add a shake animation. Boundary results (4 or 6) trigger a golden glow pulse. Commentary lines are stored as arrays of 4-5 strings per outcome in gameConfig.js, with a random index selected at shot time for variety.

3.5 Ball Tracker (Scoreboard.jsx)

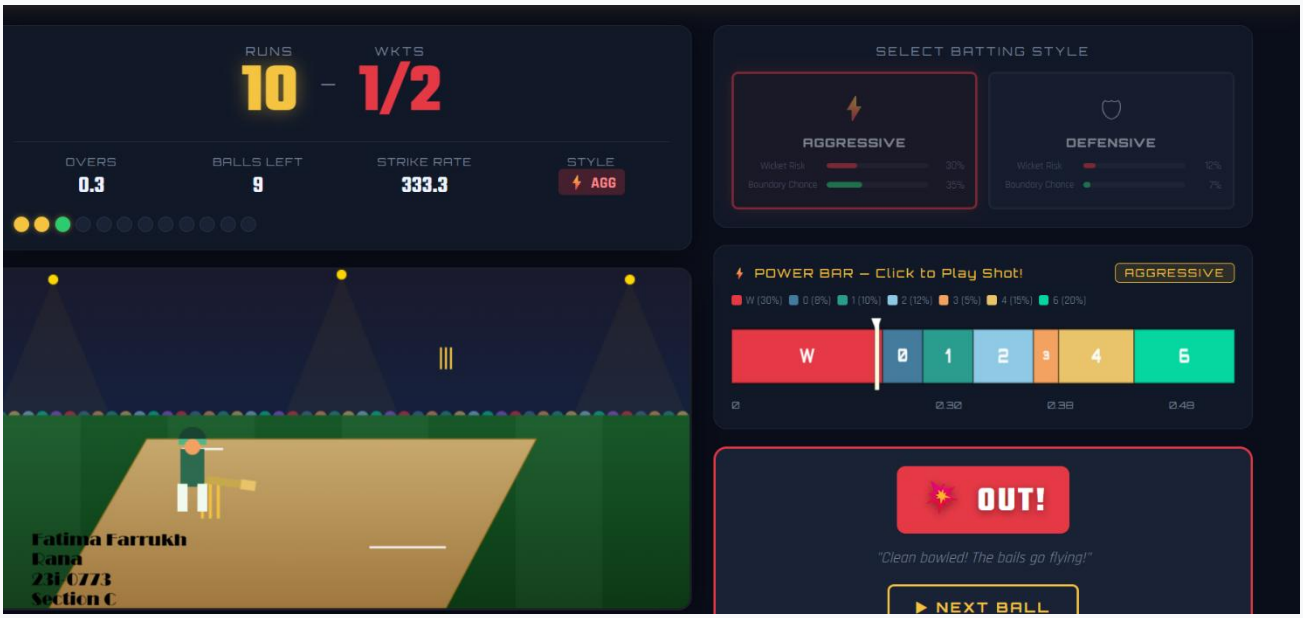
Each bowled ball lights up a dot in the scoreboard ball tracker row using a CSS transition and a pulse keyframe (pulse-dot). This gives live visual feedback of innings progress without any additional state management overhead.

4. Gameplay Screenshots

Screenshot 1 — Aggressive Batting in Action

Aggressive Batting Style Selected

Shows power bar active with Aggressive style — high-risk segments visible

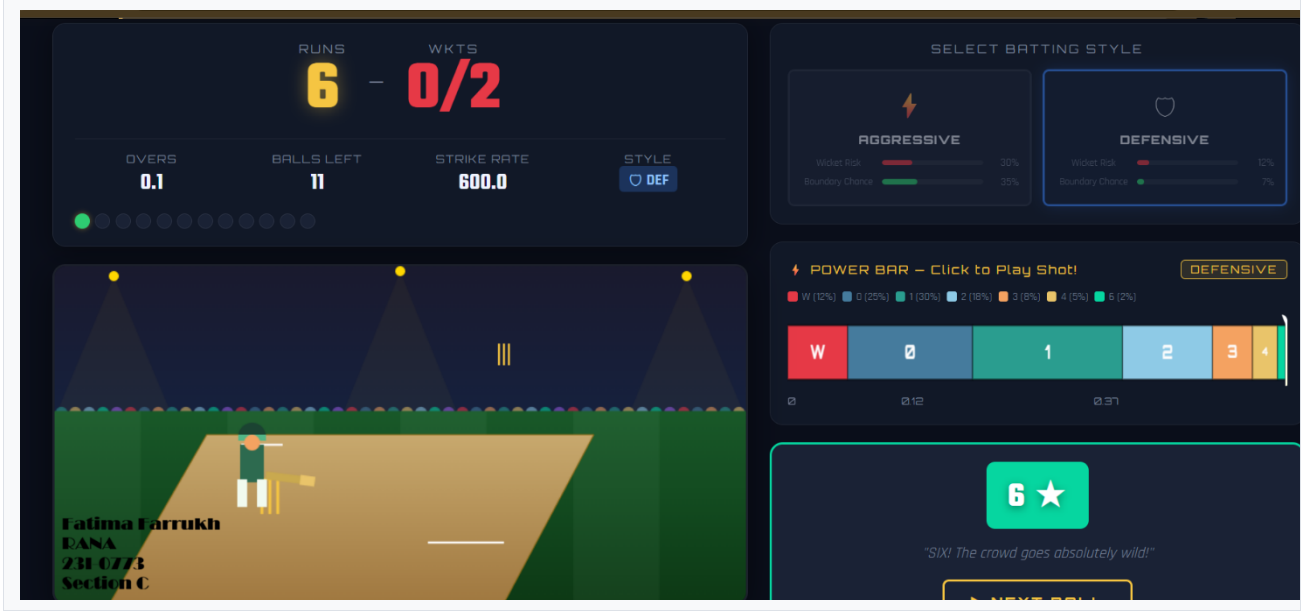


r screenshot here 1

Screenshot 2 — Defensive Batting in Action

Defensive Batting Style Selected

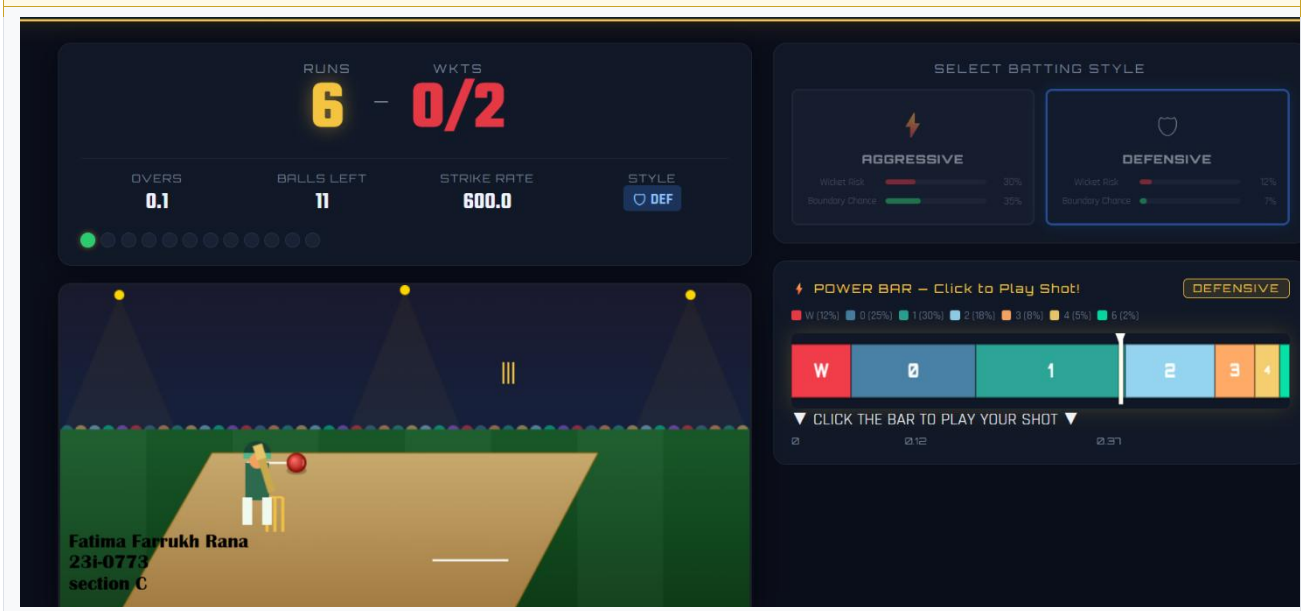
Shows power bar active with Defensive style — larger 0/1 run segments visible



Screenshot 3 — Power Bar During Gameplay

Power Bar Visible with Moving Slider

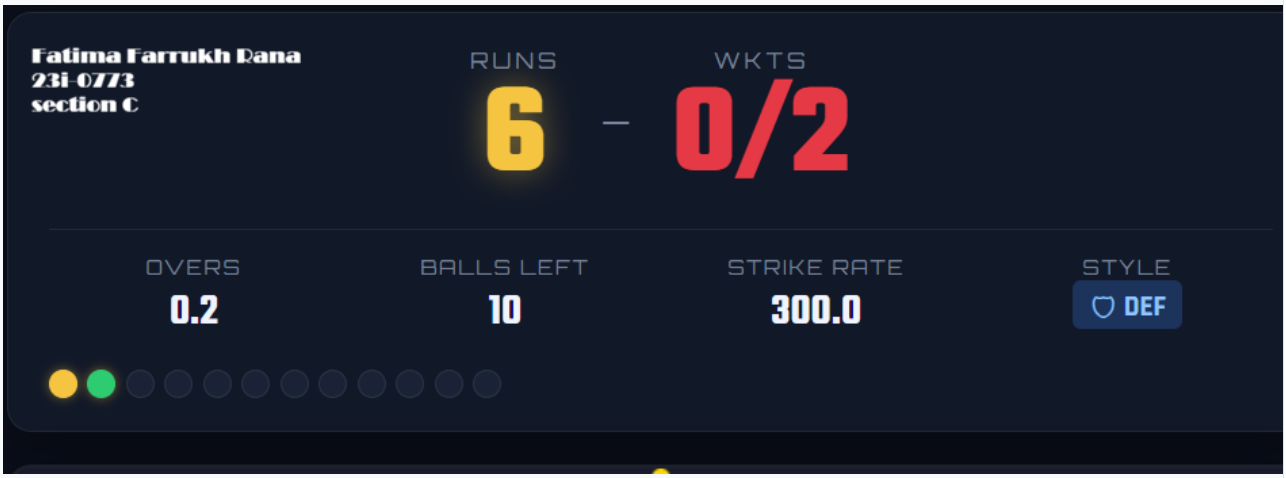
A Clear view of all coloured probability segments and the white slider in motion



Screenshot 4 — Scoreboard Updates

Scoreboard After Multiple Balls

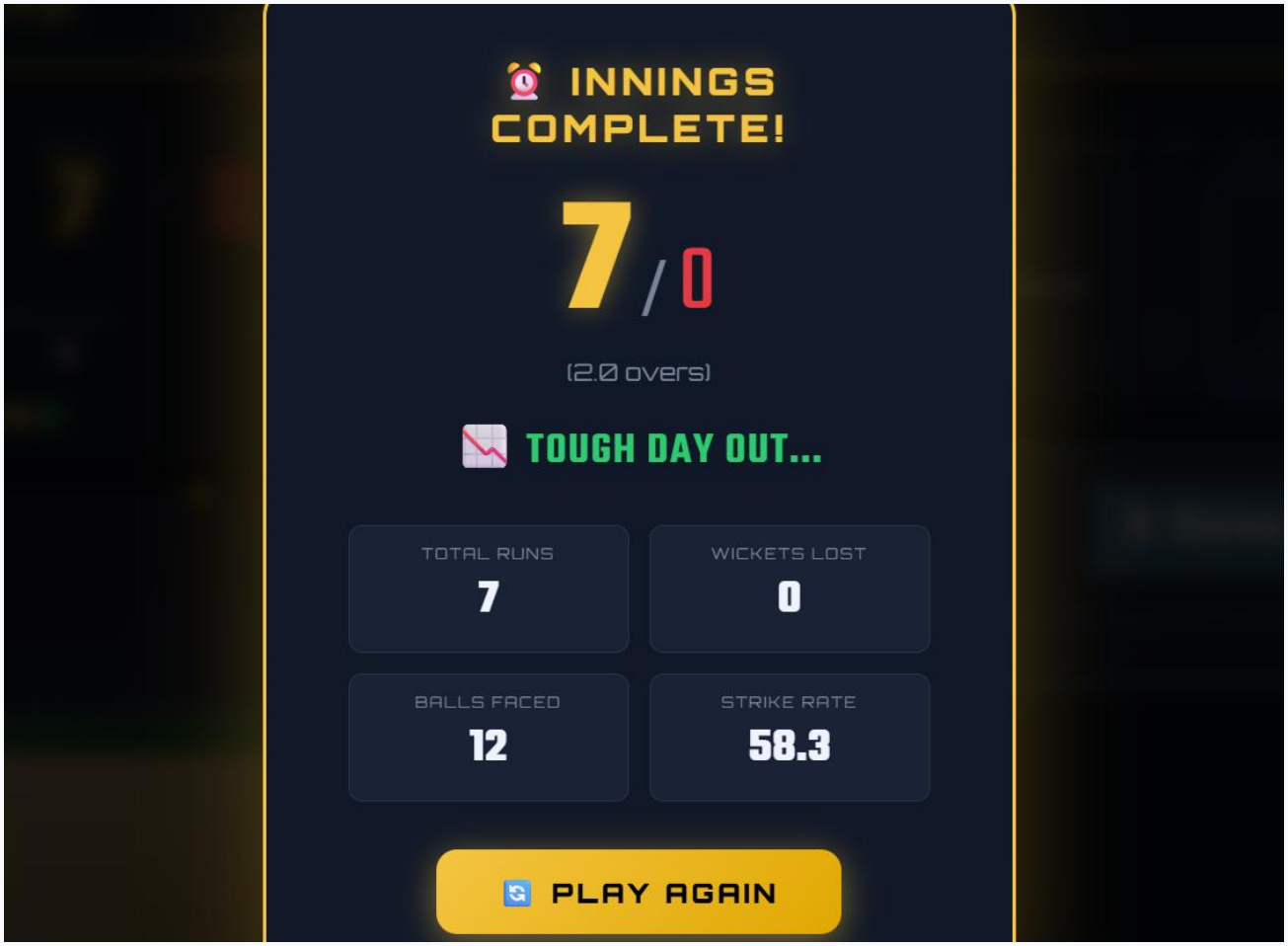
Shows updated runs, wickets, overs, strike rate, and ball tracker dots filled in



Screenshot 5 — Game Over Screen

Final Game Over / Innings Complete Screen

Shows total score, wickets, overs, strike rate, and innings rating


INNINGS COMPLETE!
7/0
(2.0 overs)
TOUGH DAY OUT...
TOTAL RUNS
7
WICKETS LOST
0
BALLS FACED
12
STRIKE RATE
58.3
PLAY AGAIN

5. Project Structure & Technologies

5.1 File Structure

File	Purpose
`src/gameConfig.js`	Probability distributions, commentary, constants, helper functions
`src/hooks/useGameState.js`	Custom React hook — all game state, phase transitions, scoring logic
`src/App.jsx`	Root component — game flow controller, layout, phase routing
`src/App.css`	All styles — dark stadium theme, animations, responsive layout
`src/components/CricketField.jsx`	Canvas 2D field, bowling animation, batting animation
`src/components/Scoreboard.jsx`	Live scoreboard with runs, wickets, overs, ball tracker
`src/components/PowerBar.jsx`	Canvas probability bar, animated slider, click detection
`src/components/BattingStyleSelector.jsx`	Aggressive / Defensive toggle with stat previews
`src/components/ResultDisplay.jsx`	Shot outcome badge with commentary and animations
`src/components/GameOverScreen.jsx`	End-of-innings summary with rating system

5.2 Technologies Used

- React 18 — Component-based UI, useState, useEffect, useCallback, useRef hooks
- HTML5 Canvas API — 2D field rendering, ball/batsman/bat sprites, animations
- CSS3 — Animations, keyframes, CSS variables, dark theme, responsive layout
- JavaScript ES6+ — Game logic, probability engine, requestAnimationFrame loops
- Vite — Development server and build tool

5.3 Key Design Decisions

1. Refs over State for animation — sliderRef.current is used instead of useState to avoid React re-renders during 60fps animation loops.
2. Phase-based game flow — A string phase variable (idle/bowling/batting/result/gameover) drives all conditional rendering, keeping logic predictable.
3. Zero randomness in outcomes — The getOutcome() function does a simple range check against cumulative segment boundaries. No Math.random() for shot results.
4. Modular components — Each UI concern is isolated in its own file with clear props interface, satisfying the separation of concerns requirement.